

YourStatGuide: An AI-Powered Statistics Learning Platform

Priyanshu Kumar,
Indian Institute of Technology Patna
Sahil,
Meghnad Saha Institute of Technology

Project Guide / Mentor Name:
Bidisha Dobe

Period of Internship: 21st January 2026 – 31st March 2026

Report submitted to: IDEAS – Institute of Data Engineering,
Analytics and Science Foundation, ISI Kolkata

GitHub Repository: github.com/Priyanshukr985/ISI_PROJECT

March 2026

Abstract

This report describes **YourStatGuide**, an AI-powered statistics learning platform developed as part of the internship work. The main aim of the project was to bring together different study activities such as theory clarification, practice, note support, and visualization into one web application.

The main question-answering part of the system is based on a Retrieval-Augmented Generation (RAG) pipeline. The PDF edition of *Fundamentals of Mathematical Statistics (A Modern Approach)* by S.C. Gupta and V.K. Kapoor is used as the main knowledge source. The document is loaded, split into chunks, converted into embeddings, and stored in FAISS for retrieval. When the user asks a question, relevant chunks are retrieved and used by the language model to generate a short and structured answer.

Along with the Ask AI module, the project also includes concept support, practice support, notes, visualizations, educational video suggestions, mathematical rendering through MathJax, and user authentication. Overall, the project was built as a practical learning tool for mathematical statistics.

Contents

Abstract	i
1 Introduction	1
1.1 Background	1
1.2 Motivation	1
1.3 Problem Statement	2
1.4 Project Objective	2
1.5 Organization of the Report	2
1.6 Training Topics	3
2 Project Objectives and Background Material	4
2.1 Natural Language Processing in the Project	4
2.2 Embeddings and Semantic Representation	4
2.3 Chunking and Overlap	5
2.4 Vector Database and Similarity Search	6
2.5 Retrieval-Augmented Generation	6
2.6 Mathematical Rendering	6
2.7 Statistical Visualization as a Learning Tool	7
3 Methodology	8
3.1 Methodology	8
3.1.1 Requirement Analysis	8
3.1.2 Source Knowledge Preparation	8
3.1.3 Text Chunking	8
3.1.4 Embedding and Index Construction	9

3.1.5	Retriever Design	9
3.1.6	Relevance Filtering and Query Rewriting	9
3.1.7	Answer Generation	9
3.1.8	Authentication Method	9
3.1.9	Notes Workflow	9
3.1.10	Visualization Workflow	10
3.1.11	Video Integration	10
3.2	System Architecture	10
3.2.1	Presentation Layer	10
3.2.2	Application Layer	10
3.2.3	Intelligence Layer	11
3.3	Technology Stack	12
4	Data Analysis and Results	13
4.1	Implementation Overview	13
4.2	Tools and Modules Used	13
4.3	Core Workflow and Processing Pipeline	14
4.3.1	Source Knowledge	14
4.3.2	Chunking	14
4.3.3	Vector Storage	14
4.3.4	Retriever and Workflow	15
4.4	Access Control and Authentication	15
4.5	System Outputs and Functional Modules	15
4.5.1	Ask AI Module	15
4.5.2	Concepts Module	15
4.5.3	Practice Module	16
4.5.4	Visualizations Module	16
4.5.5	Notes Module	16
5	Analytical and Mathematical Discussion	17
5.1	Why Moderate Chunking Was Used	17
5.2	Intuition Behind Vector Retrieval	17

5.3	Role of Query Rewriting	17
5.4	Mathematical Rendering and Readability	18
5.5	Statistical Computation in the Visualization Module	18
5.6	Illustrative Graphs Relevant to the Platform	18
6	Conclusion, Limitations, and Future Scope	21
6.1	Conclusion	21
6.2	Limitations	21
6.3	Future Scope	22
	References	24
	Appendix	25

Chapter 1

Introduction

1.1 Background

This project was built as a practical statistics learning platform. The idea was to bring together theory support, notes, practice, and visualizations in one place instead of depending on separate tools for each task.

The main question-answering part of the system is connected to a source textbook, so the user can ask statistics questions and get answers in a more structured way. Along with this, the platform also includes notes, concept support, practice support, and graph-based learning features.

1.2 Motivation

The motivation behind this work was to build a simple and useful learning tool where a student can do multiple study-related tasks in one place:

- ask theory questions from a known source,
- explore important statistical concepts,
- practice topic-wise questions,
- revise through structured notes,
- inspect graphs and statistical plots,
- and use a guided learning workflow rather than a single isolated chatbot.

1.3 Problem Statement

The project is based on a simple observation that statistics learning usually involves multiple activities. A student may need:

- theory clarification,
- topic-wise practice,
- formula support,
- note-based revision,
- and graph-based understanding.

Instead of handling these separately, this project brings them together into one interface. The main focus is on making the learning process more organized and easier to use.

1.4 Project Objective

The main objectives of the project are:

- to build a web-based platform for learning statistics,
- to create a RAG-based chatbot using *Fundamentals of Mathematical Statistics (A Modern Approach)* as the main source,
- to support concept-wise learning and topic-wise practice,
- to provide notes, visualizations, and mathematical rendering in the same platform,
- and to make the overall study workflow more organized and easier to use.

1.5 Organization of the Report

The report is organized as follows:

- Chapter 1 introduces the project and its purpose.
- Chapter 2 presents project objectives and background material relevant to the work.

- Chapter 3 explains methodology, workflow, and system architecture.
- Chapter 4 discusses implementation details and system outputs.
- Chapter 5 presents mathematical discussion, analysis, and result-oriented interpretation.
- Chapter 6 summarizes limitations, future scope, and conclusion.

1.6 Training Topics

The internship included exposure to the following training topics:

- Basic Statistics for Data Science
- Data Visualization
- Introduction to GitHub and Cloud Computing
- Streamlit - 1 (Widget handling)
- Streamlit - 2 (database integration)
- Machine Learning 1 (regression)
- Machine Learning 2 (classification)
- LLM Fundamentals
- Communication Skills
- Deep Learning 1
- Deep Learning 2
- Computer Vision Overview
- Agentic AI
- System view of DS

Chapter 2

Project Objectives and Background Material

2.1 Natural Language Processing in the Project

Natural Language Processing (NLP) is used in several parts of the system, not only in the final answer generation step. In this project, NLP is involved in:

- extracting text from the source PDF,
- splitting the document into manageable chunks,
- converting text into semantic vector embeddings,
- retrieving semantically relevant chunks for a query,
- rewriting or refining the user question when needed,
- and generating readable, exam-oriented answers.

So, NLP in this project is used as a full processing pipeline and not just as a chatbot front-end.

2.2 Embeddings and Semantic Representation

In a basic keyword-based search system, a query mostly retrieves text with matching words. This is not always enough for a learning platform because a user may ask the same idea in a different way from the book. For example, the wording used by the student may not exactly match the wording used in the textbook. That is why a semantic search approach is more useful here.

To support semantic retrieval, each text chunk is mapped to a dense vector representation. If the embedding of a user query is denoted by $\mathbf{q}$ and the embedding of the i -th document chunk is denoted by $\mathbf{d}_i$, then retrieval is based on similarity in vector space rather than exact keyword overlap.

A standard similarity measure is cosine similarity:

$$\text{cosine_sim}(\mathbf{q}, \mathbf{d}_i) = \frac{\mathbf{q} \cdot \mathbf{d}_i}{\|\mathbf{q}\| \|\mathbf{d}_i\|}. \quad (2.1)$$

If this value is high, the query and chunk are close in meaning. In this project, HuggingFace embeddings are used to create these vector representations.

2.3 Chunking and Overlap

The source PDF is not embedded as one single block. Instead, it is split into smaller overlapping chunks. This is useful because:

- one large block would be too broad and difficult to retrieve precisely,
- very small blocks may lose meaning and context,
- and overlapping chunks help preserve continuity across boundaries.

The current chunking configuration is:

- `chunk_size = 1000`
- `chunk_overlap = 200`

If chunk 1 spans characters 1 to 1000, then chunk 2 may begin at character 801. This means the last 200 characters of one chunk are repeated in the next. This helps when a formula or theorem statement starts near the end of one chunk and continues into the next.

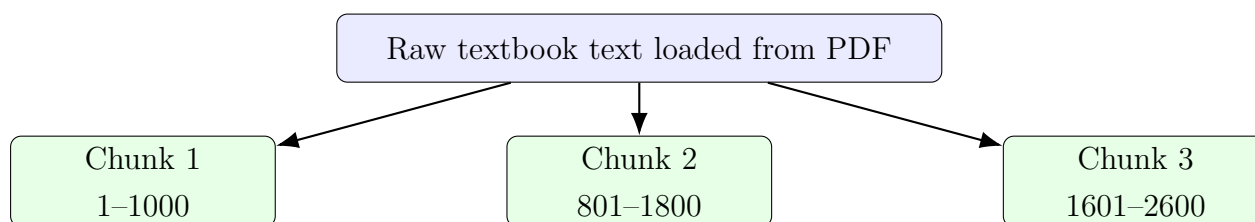


Figure 2.1: Illustration of overlapping chunking used for the source textbook.

2.4 Vector Database and Similarity Search

After chunking and embedding, all chunk vectors are stored in a vector database. In this project, FAISS is used as the vector store. The role of FAISS is to efficiently search among a large set of high-dimensional vectors and return the most similar chunks for a user query.

If the system retrieves the top k most similar chunks, then it is effectively selecting the set

$$\{\mathbf{d}_{(1)}, \mathbf{d}_{(2)}, \dots, \mathbf{d}_{(k)}\}$$

such that these vectors are the nearest to the query vector $\mathbf{q}$ according to the selected similarity measure. In the current system, a small number of high-signal chunks are retrieved so that the downstream generator receives focused context rather than an overly noisy prompt.

2.5 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) combines two stages:

1. retrieve relevant source material,
2. generate a human-readable answer from that retrieved context.

The advantage of this design is that the answer is guided by the source document instead of depending only on model memory. This reduces the chance of unsupported output and keeps the system tied to the chosen statistics book.

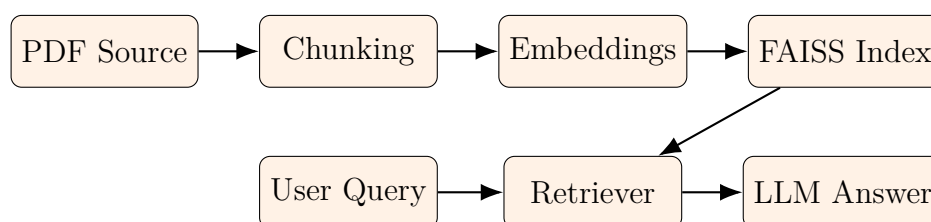


Figure 2.2: High-level Retrieval-Augmented Generation pipeline.

2.6 Mathematical Rendering

Statistics answers often require formulas, symbols, and notation. Plain text representations of mathematical expressions are harder to read and may be ambiguous. Therefore,

the system uses MathJax so that expressions such as

$$\bar{X} = \frac{1}{n} \sum_{i=1}^n X_i$$

and

$$Z = \frac{\bar{X} - \mu}{\sigma/\sqrt{n}}$$

can be rendered clearly in the browser. This is especially useful for distribution definitions, test statistics, estimators, and limit theorem statements.

2.7 Statistical Visualization as a Learning Tool

Statistics is not only symbolic but also visual. A histogram gives information about distributional shape, a box plot reflects spread and outliers, and a scatter plot suggests linear or nonlinear association. The project therefore includes a visualization module that combines graph generation with basic interpretation. This supports learning by connecting formulas, numerical summaries, and graphical intuition.

Chapter 3

Methodology

3.1 Methodology

3.1.1 Requirement Analysis

The first development stage was requirement identification. The intended user was a statistics learner who requires:

- theory clarification,
- formula support,
- practice assistance,
- note generation,
- visual interpretation,
- and a guided, low-friction workflow.

3.1.2 Source Knowledge Preparation

The textbook *Fundamentals of Mathematical Statistics (A Modern Approach)* by S.C. Gupta and V.K. Kapoor, stored locally as `Stat_Book.pdf`, was selected as the primary knowledge source for the RAG pipeline. The document is loaded and transformed into document objects suitable for later chunking and embedding.

3.1.3 Text Chunking

The loaded text is split using `RecursiveCharacterTextSplitter` with overlapping chunks. This choice balances local context preservation with retrieval precision.

3.1.4 Embedding and Index Construction

Each chunk is converted to a dense semantic vector using HuggingFace embeddings. These vectors are stored in FAISS, which allows efficient similarity search at query time.

3.1.5 Retriever Design

When the user asks a question, the system embeds the query and retrieves a small set of relevant chunks from FAISS. This set forms the evidence passed to the answer generator.

3.1.6 Relevance Filtering and Query Rewriting

The workflow contains a relevance grading stage and a query rewriting stage. These are useful when the initially retrieved content is too weak, too broad, or affected by spelling mistakes in the query. The rewriter attempts to improve the wording of the question while preserving its meaning.

3.1.7 Answer Generation

The final answer is generated by the LLM after considering the retrieved context. The output is formatted in a structured exam-style layout with:

- Definition,
- Mathematical Form,
- Key Points.

3.1.8 Authentication Method

The platform uses session-based access control. Users must sign in or sign up before reaching the protected application workspace. Passwords are stored in hashed form rather than plain text.

3.1.9 Notes Workflow

The notes module uses prompt-driven structured generation. The notes are designed to be revision-friendly and can be exported as PDFs.

3.1.10 Visualization Workflow

The visualization module accepts user-supplied parameters, generates plots using Matplotlib and Seaborn, and returns both the image and its interpretation.

3.1.11 Video Integration

Educational videos are fetched using YouTube search logic and presented within the application as supplementary learning support.

3.2 System Architecture

The architecture of the system can be divided into three layers.

3.2.1 Presentation Layer

This includes:

- landing page,
- authentication pages,
- dashboard,
- tab-based learning interface,
- rendered answers, notes, and plots.

3.2.2 Application Layer

This layer is implemented in Flask and includes:

- route handling,
- session management,
- request validation,
- note and PDF generation,
- visualization generation,
- and module-specific API responses.

3.2.3 Intelligence Layer

This includes:

- PDF loading,
- chunking,
- embeddings,
- FAISS indexing,
- retrieval,
- grading,
- query rewriting,
- and final answer generation.

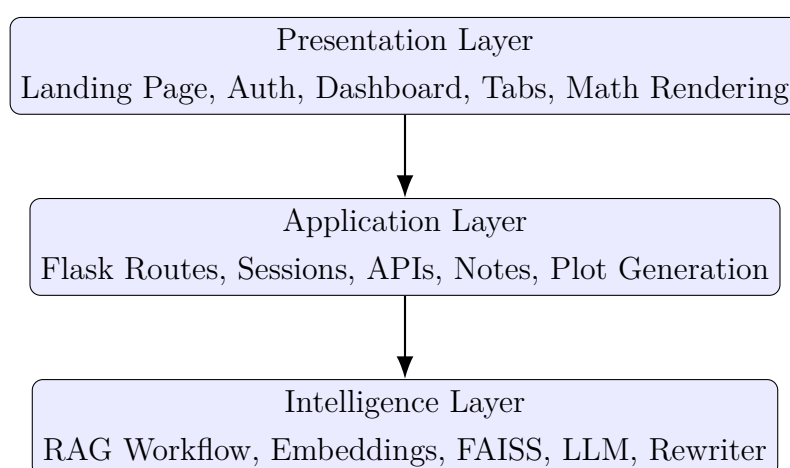


Figure 3.1: Layered architecture of the YourStatGuide platform.

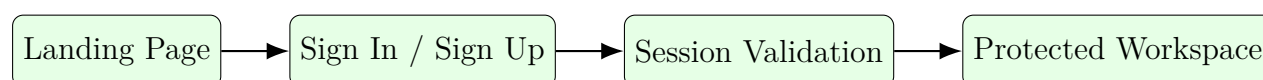


Figure 3.2: Authentication flow used to gate access to the main application.

3.3 Technology Stack

Component	Technology Used
Frontend	HTML, CSS, JavaScript
Backend	Flask
AI Workflow	LangChain, LangGraph
Language Model	Groq LLM
Embeddings	HuggingFace embeddings
Vector Database	FAISS
Visualization	Matplotlib, Seaborn
Math Rendering	MathJax
Video Integration	YouTube Data API
Authentication	Flask session, Werkzeug password hashing

Chapter 4

Data Analysis and Results

4.1 Implementation Overview

The main backend logic is implemented in `app.py`. This file initializes the Flask application and manages:

- authentication routes,
- Ask AI route,
- concept-related routes,
- practice routes,
- notes routes,
- visualization routes,
- and helper functions for note generation and plotting.

4.2 Tools and Modules Used

The frontend is organized through the following files:

- `templates/index.html` for the main application structure,
- `templates/landing.html` for the landing page,
- `templates/auth.html` for sign in and sign up pages,
- `static/css/app.css` for application styling,
- `static/css/auth.css` for authentication and landing page styling,

- `static/js/app.js` for interaction logic and API communication.

This separation improves maintainability compared to a monolithic single-template implementation.

4.3 Core Workflow and Processing Pipeline

The RAG subsystem is implemented through:

- `src/vectorstore/vector.py`
- `src/vectorstore/index_pipeline.py`
- `src/graph_builder.py`
- `src/graph_node.py`
- `src/retrieval_grader.py`
- `src/question_rewriter.py`
- `src/rag_generator.py`

4.3.1 Source Knowledge

The source document is *Fundamentals of Mathematical Statistics (A Modern Approach)* by S.C. Gupta and V.K. Kapoor, stored in the project as `Stat_Book.pdf`. This textbook is processed once and indexed into FAISS.

4.3.2 Chunking

Chunking is currently performed using:

- chunk size = 1000,
- chunk overlap = 200.

4.3.3 Vector Storage

The vector index is stored locally in:

- `faiss_index/index.faiss`
- `faiss_index/index.pkl`

4.3.4 Retriever and Workflow

On each query, the system:

1. retrieves relevant chunks from FAISS,
2. grades their usefulness,
3. optionally rewrites the query,
4. and generates the final answer.

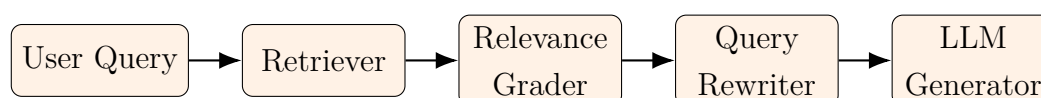


Figure 4.1: Workflow used by the Ask AI RAG module.

4.4 Access Control and Authentication

Authentication uses:

- a public landing page,
- sign in and sign up routes,
- session-based access control,
- local user storage in JSON format,
- hashed passwords through Werkzeug.

4.5 System Outputs and Functional Modules

4.5.1 Ask AI Module

This is the main theory-support chatbot. It is connected to the RAG workflow and answers statistics questions using the indexed textbook along with the exam-oriented answer format. The answer may also include source page references for transparency.

4.5.2 Concepts Module

The Concepts module presents key statistical topics as cards. A selected concept can trigger short explanations, topic-wise summaries, and supporting videos.

4.5.3 Practice Module

The Practice module supports topic-specific educational interaction. It is designed to help the learner ask for a question, a hint, or a more guided solution path in a structured way.

4.5.4 Visualizations Module

The Visualizations module allows the user to generate and inspect statistical plots such as histogram, box plot, scatter plot, normal curve, bar chart, pie chart, and Pareto distribution. The module returns both the graph and an interpretation.

4.5.5 Notes Module

The Notes module generates structured notes for selected sections and subtopics. It supports note preview, video resources, follow-up questions, and PDF export for revision.

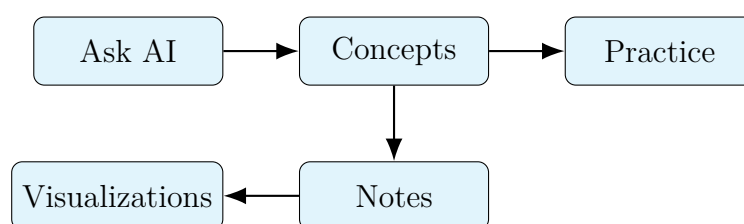


Figure 4.2: Major learning modules available within the platform.

Chapter 5

Analytical and Mathematical Discussion

5.1 Why Moderate Chunking Was Used

The chunking configuration was chosen as a practical balance between context preservation and retrieval precision. If chunks are too small, a definition and its formula may be separated. If chunks are too large, retrieval becomes broad and noisy. The current setting of 1000 characters with 200 overlap is a strong baseline because it usually keeps a theorem statement, notation, and short explanation within nearby chunks.

5.2 Intuition Behind Vector Retrieval

Suppose a user asks a question represented by vector $\mathbf{q}$ and each text chunk is represented by a vector $\mathbf{d}_i$. Retrieval effectively ranks chunks by similarity score in embedding space. In this project, the embeddings are normalized before indexing, so the retrieval can be understood in cosine-similarity terms. A common similarity expression is

$$\frac{\mathbf{q} \cdot \mathbf{d}_i}{\|\mathbf{q}\| \|\mathbf{d}_i\|}.$$

This means the system is not only comparing words but also comparing semantic direction in vector space.

5.3 Role of Query Rewriting

User queries are not always clean. A learner may type short, vague, or misspelled questions. The query rewriter attempts to preserve meaning while improving retrieval qual-

ity. This is especially helpful when a query contains spelling mistakes or under-specified phrasing.

5.4 Mathematical Rendering and Readability

Formulas are rendered using MathJax so that the mathematical form of a concept is clear. For example, the sample mean is displayed as

$$\bar{X} = \frac{1}{n} \sum_{i=1}^n X_i$$

instead of as a plain-text approximation. This improves readability and reduces ambiguity in statistical notation.

5.5 Statistical Computation in the Visualization Module

The visualization module is not purely decorative. It uses standard statistical quantities to help interpret plots. Examples include:

- sample mean:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i,$$

- sample variance:

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2,$$

- normal density:

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right).$$

The module therefore supports both visual output and statistical interpretation.

5.6 Illustrative Graphs Relevant to the Platform

The platform currently supports multiple visualization types, including histogram, box plot, scatter plot, normal curve, bar chart, pie chart, and Pareto distribution. The following figures show a few representative examples of the kinds of graphs used in the platform.

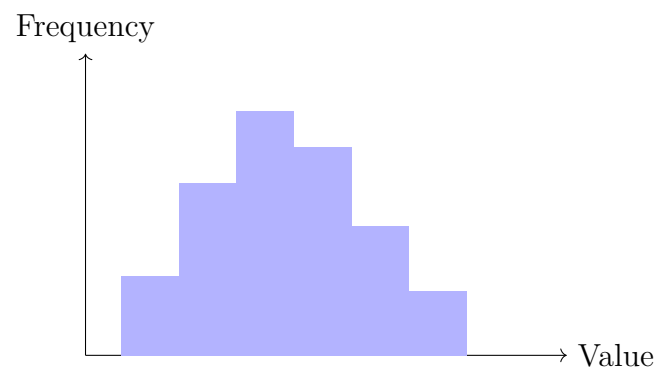


Figure 5.1: Representative histogram-style figure for frequency distribution learning.

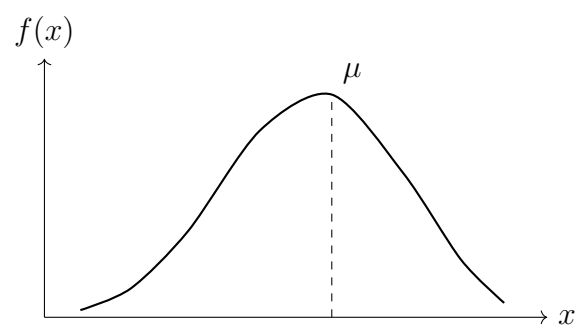


Figure 5.2: Representative bell-shaped curve for discussing the normal distribution.

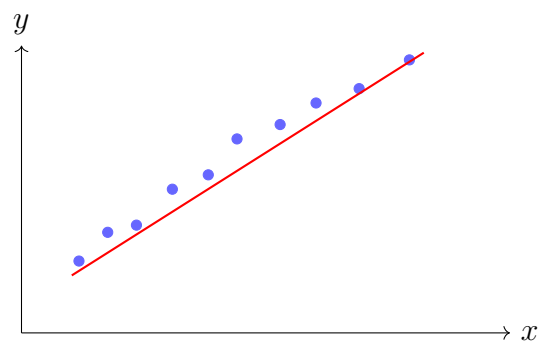


Figure 5.3: Representative scatter plot with positive association.

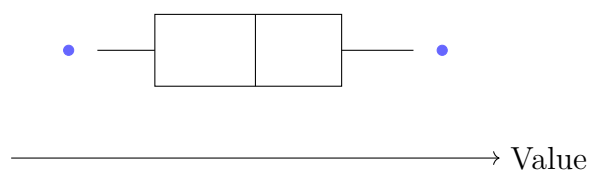


Figure 5.4: Representative box plot showing spread, median, and possible outliers.

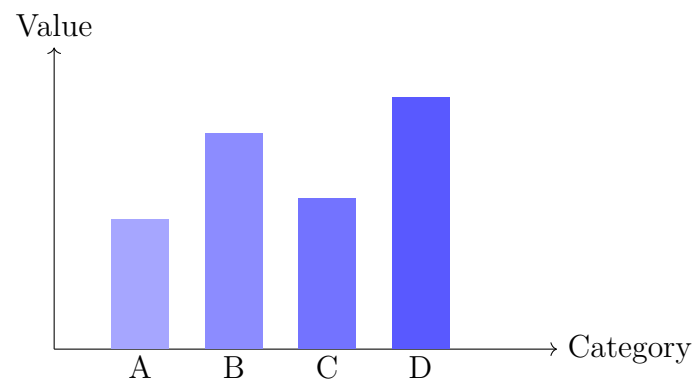


Figure 5.5: Representative bar chart for category-wise comparison.

Chapter 6

Conclusion, Limitations, and Future Scope

6.1 Conclusion

This project resulted in the development of an AI-powered statistics learning platform that combines question answering, concept support, practice, notes, and visualizations in one place. The system is centered around a RAG-based workflow and uses a statistics textbook as its main knowledge source.

The project is useful as a practical learning tool and also helped in understanding how retrieval, embeddings, mathematical rendering, and web development can be combined in a single application. With further improvement, the platform can be extended into a more polished and robust learning system.

6.2 Limitations

The current system still has some limitations:

- the frontend can be modularized further,
- `app.py` contains many responsibilities in one file,
- local JSON-based authentication,
- retrieval quality depends on chunk quality and embedding performance,
- and full automated testing is not yet extensive.

6.3 Future Scope

Possible future improvements include:

- modular route and service separation in the backend,
- database-backed authentication,
- progress tracking and performance analytics,
- richer note formatting and better PDF styling,
- evaluation and tuning of retrieval quality,
- exploration of page-aware or hierarchical retrieval alternatives such as PageIndex,
- stronger testing of answer correctness and route behavior,
- and deployment with more features.

Discussion: PageIndex as a Possible Retrieval Improvement

An interesting future direction for retrieval is the use of **PageIndex**, an open-source framework by VectifyAI for *vectorless, reasoning-based RAG*. Unlike the current chunking plus FAISS pipeline used in this project, PageIndex organizes a long document into a hierarchical tree, similar to a machine-generated table of contents, and then performs retrieval by reasoning over that tree.

In simple terms, the difference is:

- the current system retrieves chunks by semantic vector similarity,
- whereas PageIndex attempts to retrieve information through a structured document tree and reasoning over sections and pages.

This can be useful for long academic documents because it may improve:

- page-level traceability,
- section-aware retrieval,
- citation consistency,
- and explainability of why a particular page or section was selected.

For a future version of YourStatGuide, such a framework could be explored as an alternative or complementary retrieval layer, especially if stronger page citation and section-aware navigation become important design goals.

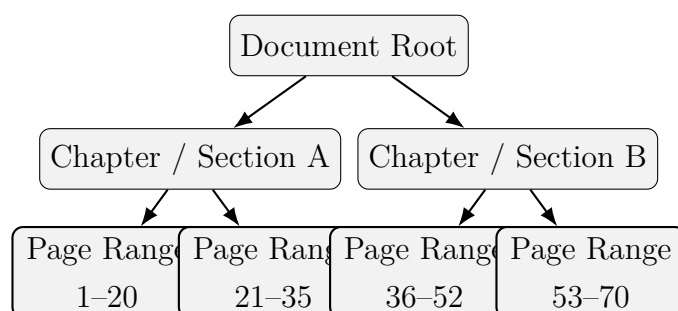


Figure 6.1: Tree-style hierarchical retrieval idea inspired by the PageIndex framework.

References

1. Pallets Projects. [Flask Quickstart](#).
2. Pallets Projects. [Werkzeug Security Helpers](#).
3. LangChain. [Python API Reference](#).
4. LangGraph. [Overview Documentation](#).
5. LangChain Community. [PyPDFLoader Documentation](#).
6. LangChain Text Splitters. [RecursiveCharacterTextSplitter Documentation](#).
7. LangChain HuggingFace. [HuggingFaceEmbeddings Documentation](#).
8. Sentence Transformers. [Official Documentation](#).
9. FAISS. [Official Documentation](#).
10. Groq. [API Reference](#).
11. Groq. [LangChain Integration](#).
12. Matplotlib. [Pyplot Tutorials](#).
13. Seaborn. [Official Tutorial](#).
14. MathJax. [Official Documentation](#).
15. Google Developers. [YouTube Data API Overview](#).
16. VectifyAI. [PageIndex GitHub Repository](#).
17. PageIndex. [Official Documentation](#).

Appendix

Appendix A: Important Project Files

- `app.py`
- `templates/index.html`
- `templates/landing.html`
- `templates/auth.html`
- `static/css/app.css`
- `static/css/auth.css`
- `static/js/app.js`
- `src/vectorstore/vector.py`
- `src/vectorstore/index_pipeline.py`
- `src/graph_builder.py`
- `src/graph_node.py`
- `src/rag_generator.py`
- `src/retrieval_grader.py`
- `src/question_rewriter.py`
- `src/video_search.py`
- `src/llm_model.py`
- `Stat_Book.pdf` — *Fundamentals of Mathematical Statistics (A Modern Approach)*
by S.C. Gupta and V.K. Kapoor
- `faiss_index/index.faiss`
- `faiss_index/index.pkl`

Appendix B: Notes on the Current Implementation

- The platform currently uses local JSON-based user storage for authentication.
- The RAG pipeline is built on a precomputed FAISS index generated from *Fundamentals of Mathematical Statistics (A Modern Approach)* by S.C. Gupta and V.K. Kapoor.
- Mathematical expressions are rendered in the browser using MathJax.
- Statistical plots are generated on the backend and returned to the frontend as image outputs.
- GitHub repository for the project: https://github.com/Priyanshukr985/ISI_PROJECT